

Capstone project

Google Africa Data Engineering Bootcamp

Document Owner	Wizeline
Last Updated	19/06/2023
Author	Data Engineering Team
Reviewer	Marco Mendoza

Contents

Capstone Project: End to End Solution	3
Expected Outcome	4
Problem Description	4
Project Timeline	6
Data Pipeline - The end-to-end solution	7

Capstone Project: End to End Solution

This project does not require implementing a specific technology stack; you can solve it based on the techniques and best practices reviewed during the Bootcamp.

The following table shows examples of paths that enable engineers to build the solution; feel free to explore others.

Subject	Localhost	GCP
Orchestration	Airflow	Airflow
Data Processing	Spark	Dataflow
Storage	PostgreSQL / MySQL / File System	GCS, Cloud SQL, BigQuery
Intermediate Steps	MySQL / Jupyter Notebooks	
CI/CD	Bash Shell	Terraform
Serverless *	-	Dataflow, Scheduler Cloud API

** Optional step

Table 1. Examples of Architectures

Expected Outcome

The goal is to create an end-to-end solution; this problem reflects a real-world scenario where a Data Engineer must configure the environment and resources, integrate the data from external sources, clean it, and process it for further analysis. When the data is ready, transform it to reflect business rules and deliver knowledge in the form of insights.

Problem Description

Assume you work for a user behavior analytics company that collects user data and creates user profiles. You must build a data pipeline to populate the `fact_movie_analytics` table, an OLAP table. The data from `fact_movie_analytics` is helpful for analysts and tools like dashboard software.

The table `fact_movie_analytics` takes information from:

- A PostgreSQL table named `user_purchase`. You will find in the Google Classroom the file to fill this table.
- Daily data by an external vendor in a CSV file named `movie_review.csv` that populates the `classified_movie_review` table. This file contains a customer id, a review id, and the message from the movie review. You will find the file in the Google Classroom.
- Daily data by an external vendor in a CSV file named `log_reviews.csv`. This file contained the id review and the metadata about the session when the movie review was done, like log date, device (mobile, computer), OS (windows, Linux), region, browser, IP, and phone number. You will find the file in the Google Classroom.

The following image provides a visual of the tables and their relationships:

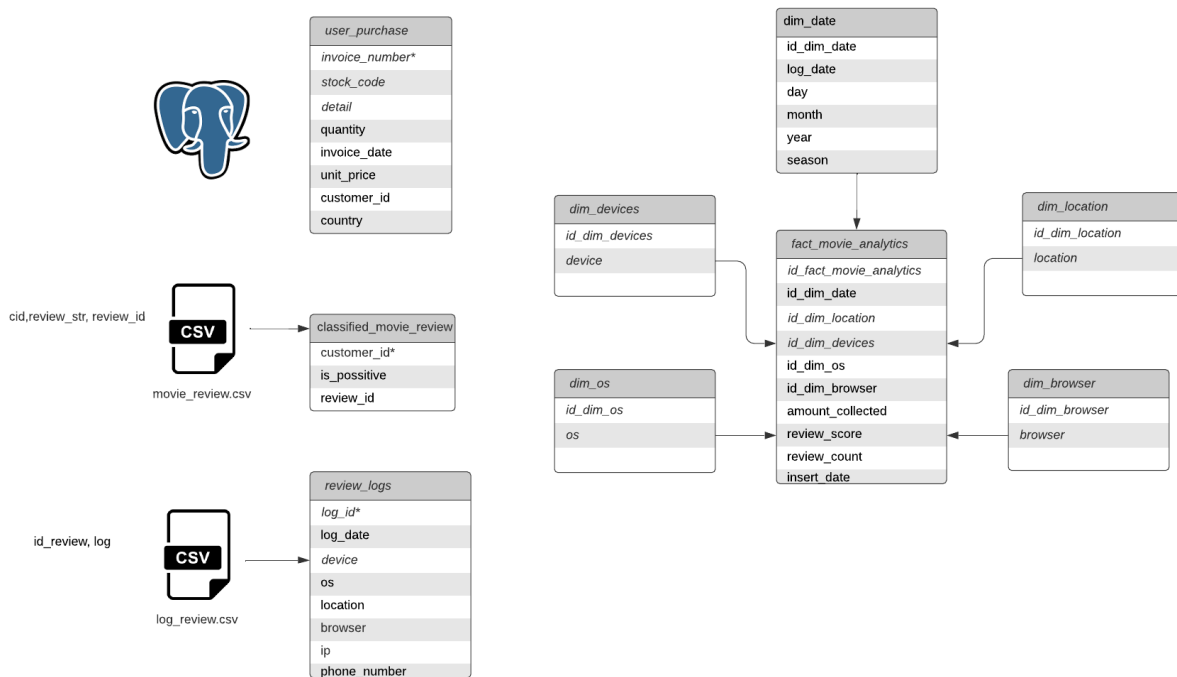


Figure 1. Data Structure

Project Timeline

Milestone / Goal	Description	Bootcamp Module
1 - Airflow Service in the Cloud	Have a running Airflow instance (latest stable release) on any Cloud provider; the instance can be implemented as a managed service or as a Dockerized application or use the provided Terraform Template. All options are valid.	2
2 - Database connection with Airflow	Create a secure, persistent connection from your implemented Airflow instance to any Cloud Database Instance. You must implement a Directed Acyclic Graph (DAG) to populate Database Instance.	3
3 - ETL using a pipeline with Spark	Create the jobs, and DAG steps and implement the required transformations with Spark as a supporting framework.	4
4 - Data Warehouse Management	Create a Data Warehouse structure (we are suggesting some structure options in Step 6 of the Data Pipeline - The end-to-end solution section) for an external analytics team to be able to consume such Data Warehouse entities.	5
5 - Dry Run / Presentation	Showcase and demonstrate the outcome of the end-to-end solution in a live presentation to a panel of mentors/architects. Elaborate on your problem-solving and thought process. And explain your architectural decisions.	Final

Data Pipeline - The end-to-end solution

Throughout the Bootcamp, you will review the steps for building a data pipeline. The following is a list of phases describing the process you must follow. Note that these steps are a strong reference for the design and the expected outcome, but you can include other tasks that help you achieve the solution.

1. **Orchestration:** The use of Airflow to automate all jobs in the project is mandatory.
2. **CI/CD:** Based on your cloud selection, Wizeline will provide a template to create and manage cloud resources through Terraform. You will execute this file through Airflow too.
3. **Setup the environment:** Part of the environment setup process is to create cloud buckets and a PostgreSQL DB with one table using the following schema:

```
CREATE SCHEMA <schema_name>;
CREATE TABLE <schema_name>.user_purchase (
    invoice_number varchar(10),
    stock_code varchar(20),
    detail varchar(1000),
    quantity int,
    invoice_date timestamp,
    unit_price numeric(8,3),
    customer_id int,
    country varchar(20)
);
```

4. **Load the data:** Wizeline provides three CSV files; download the files and prepare your environment accordingly. You must upload the `user_purchase.csv` file into a PostgreSQL DB and the last two CSV files (`movie_review.csv` and `log_reviews.csv`) into the cloud bucket you created in the previous step (RAW area).
5. **Transforming the data:** Merge the three datasets as follows:

For the `movie_review.csv` file:

- a. Work with the `cid` and `review_str` columns to get a list of words used by users.



Note: You can implement the `pyspark.ml.feature.Tokenizer` class to create a list of words named `review_token`.

- b. Remove stop words if needed with `pyspark.ml.feature.StopWordsRemover`.
- c. Look for data that contain the word “good”, consider the review as positive, and name it as `positive_review`.
- d. Add a timestamp when the job is running as the `insert_date` column.
- e. **Optional:** Add a more extensive list of good words and stop words.
- f. Select the appropriate file format for the next steps.
- g. Use the following logic to convert a boolean column to an integer:

```
reviews.positive_review = CASE
    WHEN positive_review IS True THEN 1
    ELSE 0
END
```



Note: You will use this value in step 8.

- h. Store your results in a new file in the STAGE area (`user_id`, `positive_review`, and `review_id`).
- i. Download the entire `<schema_name>.user_purchase` from the PostgreSQL table and store it in the STAGE area.

For the `log_reviews.csv` file:

- a. Map the structure for the DataFrame schema according to the `log_review` column that contains the XML as a string.
- b. Work with the `log` column to get all the metadata and build your columns for your DataFrame.



Note: You can use the `databricks.spark.xml` library

- c. Don't forget to drop the log column by the end
 - d. Store your results into a new file in the STAGE area (log_id, log_date, device, os, location, browser, ip, phone_number)
6. **Proposal for setup the DW environment:** Use the Terraform template to accommodate the corresponding blocks to create your DW; you will have to build your dim and fact tables according to the data structure. Use the following schema:

Fact table:

```
CREATE TABLE fact_movie_analytics (
  customerid INTEGER,
  id_dim_devices INTEGER,
  id_dim_location INTEGER,
  id_dim_os INTEGER,
  id_dim_browser INTEGER,
  amount_spent DECIMAL(18, 5),
  review_score INTEGER,
  review_count INTEGER,
  insert_date DATE);
```

Dim tables:

```
CREATE TABLE dim_date (
  id_dim_date INTEGER,
  log_date DATE,
  day VARCHAR,
  month VARCHAR,
  year VARCHAR,
  season VARCHAR
);
```

```
CREATE TABLE dim_devices (
  id_dim_devices INTEGER,
  device VARCHAR
```

```
);
CREATE TABLE dim_location (
  id_dim_location INTEGER,
  location VARCHAR
);
```

```
CREATE TABLE dim_os (
  id_dim_devices INTEGER,
  os VARCHAR
);
```

```
CREATE TABLE dim_browser (
  id_dim_devices INTEGER,
  browser VARCHAR
);
```

7. **Create IAM roles:** Ensure to use IAM roles and provide all needed privileges to let your DW read the data from the Cloud bucket.
8. **Build your dimension table and run analytic queries:** To calculate fact movie analytics, you must build your dimension table. Remember that you stored in the silver zone (STAGING area) the data from **movie_review.csv** and **log_reviews.csv**. You have to build 5 dim tables with the following dimensions: date, devices, location, os, and browser, then build your fact table **fact_movie_analytics**. Take as reference the following description of the fields for the DW table:

customerid => **user_purchase**.customerid,
 amount_spent => **SUM**(**user_purchase**.quantity * **user_purchase**.unit_price),
 review_score => **SUM**(**reviews**.positive_review),
 review_count => **COUNT**(**reviews**.id),
 insert_date => **airflow timestamp**

Next you will find some of the many metrics that you can build from your data:

- How many reviews were done in California, NY and Texas?
- How many reviews were done in California, NY, and Texas with an Apple device? And how many for each device type?
- What are the states with more and fewer reviews in 2021?
- Which device is the most used to write reviews in the East, and which one in the West?